

Deferred Execution & Execution Flow

Zephyr RTOS Intermediate Course





Quick Recap — Lecture 2



- Race conditions & shared-state hazards
- Mutexes and ownership semantics
- Semaphores and signaling patterns
- Atomic operations
- Blocking vs busy waiting
- Interrupt locking & spinlocks
- Deadlocks and concurrency design



Today's Agenda



- Why deferred execution exists
- Timers: delayed and periodic triggers
- Workqueues: moving work into thread context
- Delayable work: timer plus workqueue
- Patterns: bottom-half, debounce, periodic
- `k_poll`: one thread, many event sources
- Common failure modes



Timers



Timer Use Cases



Timers are used when work must be tied to time.

```
while (1) {  
    check_status();  
    k_sleep(K_MSEC(10));  
}
```

- Avoid polling threads for simple time tracking
- Run once after a timeout
- Run periodically
- Trigger retry or timeout handling
- Start real processing in another context



Create and Start a Timer



```
K_TIMER_DEFINE(my_timer, expiry_fn, stop_fn);
```

```
/* one-shot: fire once after 200 ms */  
k_timer_start(&my_timer, K_MSEC(200), K_NO_WAIT);
```

```
/* periodic: first fire after 200 ms, then every 200 ms */  
k_timer_start(&my_timer, K_MSEC(200), K_MSEC(200));
```

- `duration`: time to first expiry
- `period`: interval after that
- `K_NO_WAIT`: one-shot timer



Timer Utility APIs



API	Use
<code>k_timer_stop(&timer)</code>	Cancel a running timer
<code>k_timer_user_data_set(&timer, data)</code>	Attach context pointer
<code>k_timer_user_data_get(&timer)</code>	Retrieve context pointer
<code>k_timer_status_sync(&timer)</code>	Block current thread until expiry

Important: `k_timer_status_sync()` is thread context only



Timer Expiry Context



```
void my_expiry_fn(struct k_timer *timer)
{
    /* Runs in timer interrupt context. */
}
```

Allowed in expiry	Not allowed
<code>k_sem_give()</code>	<code>k_mutex_lock()</code>
<code>k_work_submit()</code>	<code>k_sleep()</code>
Atomic operations	Any blocking API
Short state update	<code>k_timer_status_sync()</code>



DEMO 1: Timer Expiry Context



The Third Execution Context



Meet the Workqueue



A workqueue is a kernel thread that processes submitted work items.

Capability	Workqueue	Dedicated Thread
Can sleep or block	Yes, keep short	Yes
Can use mutexes	Yes	Yes
Runs handlers serially	Yes	No
Stack cost	Shared queue stack	One stack per thread

- The system workqueue reuses one existing stack
- A custom workqueue still costs one thread stack
- It can still replace several dedicated threads



Workqueue Fundamentals



Key APIs:

```
K_WORK_DEFINE(name, handler)
```

```
k_work_submit(&work)
```

```
k_work_submit_to_queue(&queue, &work)
```

```
k_work_queue_start(&queue, stack, size, priority, NULL)
```

Core Properties:

- Handlers run in thread context, full kernel access
- Serialisation — items on the same queue never run concurrently
- `k_work_submit()` may be called from ISR or thread
- System or Custom workqueue



From ISR to Workqueue



```
K_WORK_DEFINE(sensor_work, sensor_handler);
```

```
void sensor_isr(void *arg)
{
    k_work_submit(&sensor_work);
}
```

```
void sensor_handler(struct k_work *work)
{
    read_and_process_data();
}
```

Important: `k_work_submit()` queues work. It does not run it immediately



Serialisation



```
submit A, B, C → [A][B][C] → handler(A) → handler(B) → handler(C)
```

- No two handlers run concurrently on the same queue
- **Mutual exclusion for free**
- Multiple queues *can* run concurrently
- Other threads and ISRs still need synchronization
- One slow handler delays the next item



Delayable Work



`k_work_delayable` combines:

- a timer
- a work item
- a workqueue handler

`[DELAYED]` → `[QUEUED]` → `[RUNNING]` → `[IDLE]`



k_work_delayable



API	Behaviour
<code>K_WORK_DELAYABLE_DEFINE(name, handler)</code>	Define a delayable work item
<code>k_work_schedule(&dwork, delay)</code>	Schedule if not already scheduled
<code>k_work_reschedule(&dwork, delay)</code>	Always reset the delay
<code>k_work_cancel_delayable(&dwork)</code>	Async cancel
<code>k_work_cancel_delayable_sync(&dwork)</code>	Sync cancel

- **Schedule:** first event wins
- **Reschedule:** last event wins



Deferred Execution Patterns



Bottom-Half



- Top half: ISR captures minimal state
- Bottom half: workqueue handler processes data

```
void uart_rx_isr(void *arg)
{
    rx_byte = uart_read_byte_reg();
    k_work_submit(&uart_rx_work);
}
```

```
void uart_rx_handler(struct k_work *work)
{
    process_byte(rx_byte);
}
```

```
K_WORK_DEFINE(uart_rx_work, uart_rx_handler);
```



Debounce



- Mechanical button contacts can bounce
- One press may generate many interrupts
- Run the handler once after a quiet period

```
K_WORK_DELAYABLE_DEFINE(button_work, button_handler);  
  
void on_button_event(void)  
{  
    k_work_reschedule(&button_work, K_MSEC(50));  
}
```

event event event event → quiet 50 ms → handler once



Schedule vs Reschedule



API	Behaviour	Use case
<code>k_work_schedule()</code>	No-op if already scheduled	First event wins
<code>k_work_reschedule()</code>	Always resets the delay	Last event wins

- First event wins: rate limiting
- Last event wins: debounce



Periodic Work



- Self-rescheduling work can replace a simple polling thread
- This is useful for approximate periodic work

```
K_WORK_DELAYABLE_DEFINE(poll_work, poll_handler);

void poll_handler(struct k_work *work)
{
    struct k_work_delayable *dwork;

    dwork = k_work_delayable_from_work(work);
    read_and_publish_sensors();
    k_work_reschedule(dwork, K_MSEC(100));
}

k_work_schedule(&poll_work, K_MSEC(100));
```



DEMO 2: Workqueue Patterns in Practice



Event-Driven Execution



`k_poll`



A single thread can wait on several independent sources.

```
UART semaphore  
SPI semaphore    → one dispatcher thread  
timer signal
```

- Without `k_poll`: one thread per source
- With `k_poll`: one sleeping thread waits on all sources
- When any source is ready, the thread wakes



k_poll

Fundamentals



- Key APIs:

```
K_POLL_EVENT_INITIALIZER(type, mode, object)
```

```
k_poll(events, num_events, timeout)
```

- Event types:

```
K_POLL_TYPE_SEM_AVAILABLE
```

 — semaphore count > 0

```
K_POLL_TYPE_FIFO_DATA_AVAILABLE
```

 — FIFO has data

```
K_POLL_TYPE_SIGNAL
```

 — poll signal was raised

- Rules:

- `k_poll` notifies only
- Call `k_sem_take()` or `k_fifo_get()` after wakeup
- Check all events on return
- More than one event may be ready



Three Ways to Be Event-Driven



Tool	Best for
Work item	Short, single-source, non-blocking processing
Semaphore + thread	Long blocking work
<code>k_poll</code>	One thread reacting to multiple sources

Question: how long does the work take, and how many sources exist?



Putting It All Together



Situation	Tool
ISR needs to defer work to thread context	<code>k_work_submit</code>
Run once after a delay	<code>k_work_schedule</code>
Debounce: last event in burst wins	<code>k_work_reschedule</code>
Periodic, approximate interval	self-rescheduling <code>k_work_delayable</code>
Periodic, strict interval	<code>k_timer</code> → <code>k_work_submit</code>
Wait on one event source	<code>k_sem_take</code> / <code>k_fifo_get</code>
Wait on multiple event sources	<code>k_poll</code> dispatcher thread
Long blocking or strict priority	dedicated thread or custom workqueue



What Can Go Wrong: Wrong Context



Problem	Symptom	Fix
Blocking in timer expiry	Fault or assertion	Submit work, return
Blocking in ISR	Fault or broken latency	Move work to thread context
Calling thread-only APIs from expiry	Unstable behavior	Use ISR-safe APIs only



What Can Go Wrong: Workqueue Latency



Problem	Symptom	Fix
Blocking in system workqueue	Unrelated handlers stop	Short handler or custom queue
Assuming submit means execute	Silent latency bugs	Remember execution is scheduled
Slow handler on shared queue	Next item delayed	Split queues or reduce handler time



What Can Go Wrong: Queue and Cancel Semantics



Problem	Symptom	Fix
Ignoring submit return	Missed event	Check return values
Bursty producer	Backlog or starvation	Short handlers and backpressure
Async cancel assumption	Handler still running	Use sync cancel when required



Homework



- **Goal:** Convert a polling loop to an event-driven workqueue architecture
- **Starter code provided:** `l3-homework`
- `sensor_sim` thread sets a flag and calls `k_sem_give` every 100 ms
- `polling_thread` checks a shared flag every 10 ms with `k_sleep`
- **Tasks:**
 - Run the starter and count wake-ups in the log
 - Replace `polling_thread` with a `k_work` handler
 - Confirm the handler runs only on real events
 - Bonus: debounce five events within 20 ms using `k_work_reschedule`
- Push tag `l3-task1`

Thank You





Resources

1. [Workqueue Threads](#)
2. [Timers](#)
3. [Polling API](#)

